

True Alignment is Impossible

Holding the Goal Versus Satisfying the User

Registry: [HOWL-LLM-11-2026]

Series Path: [HOWL-LLM-10-2026] → [HOWL-LLM-11-2026]

DOI: 10.5281/zenodo.22979243

Date: September 2026

Domain: Machine Learning / Human-AI Interaction / Software Engineering Process / Constraint Satisfaction

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.8.

Abstract

This paper makes one claim. True Alignment between a transformer's output and a requester's full goal is not reachable with the current architecture. The claim is structural. It does not depend on the quality of any model. It does not depend on the skill of any developer. The paper defines True Alignment as the output holding the requester's complete set of interacting constraints. It defines a second and weaker condition, casual alignment, as the requester accepting the output. The paper states that casual alignment is reachable and that True Alignment is not. The reason is that the requester's complete constraint set is never a full input to the process, and that the process erodes even the partial goal that was supplied. The paper does not present this as a defect to be blamed. It presents it as a property of the architecture that determines how the architecture must be used. The correct use is transformation of supplied material, not the pursuit of a goal the process cannot hold. The paper ends with the working method that follows from the claim.

1. Introduction

The word "alignment" carries two meanings that are usually not separated. One meaning is strong. It is the condition where the output holds the goal the requester actually has. The other meaning is weak. It is the condition where the requester looks at the output and accepts it. In everyday use the two meanings are treated as one. A requester who accepts an output is said to have received an aligned output.

This paper separates the two meanings and shows that they are not the same. It names the strong meaning True Alignment and the weak meaning casual alignment. It then shows that the strong meaning is not reachable with the current architecture, and that the weak meaning is reachable and useful.

The paper's position is descriptive. It states how the architecture works. It states what follows from that. It does not argue that the architecture is broken. It does not argue that anyone made an error. The paper's purpose is to identify the correct goal for this architecture, which is transformation, and to show why a different goal, True Alignment, cannot be met by structure.

Section 2 defines the terms. Section 3 states why the full goal is never an input. Section 4 states how the process erodes the partial goal that was supplied. Section 5 shows that both a small request and a large request fail to reach True Alignment, for opposite reasons. Section 6 states the correct use that follows. Section 7 gives the background data that led to the claim. Section 8 states the boundaries of the claim. Section 9 summarizes.

2. Two Kinds of Alignment

This section defines the terms once, in dependency order. Later sections use them without restatement.

2.1 Constraint. A condition that an output must satisfy to be correct for the requester's goal.

2.2 Joint constraint. The complete set of constraints that must hold at the same time for an output to be correct. The constraints in a joint constraint interact. The correct value of one depends on the correct value of others. The joint constraint cannot be satisfied one constraint at a time.

2.3 True Alignment. The condition where the output holds the requester's full joint constraint. The output is correct against every constraint in the set, and it is correct against the interactions between them.

2.4 Casual alignment. The condition where the requester accepts the output. Casual alignment is measured by the requester's reaction, not by the output's relation to the full joint constraint.

2.5 The two-side property. True Alignment and casual alignment are measured on different sides. True Alignment is a property of the output against the full goal. Casual alignment is a property of the requester's response to the output. A single output can have casual alignment without True Alignment. This happens when the requester accepts an output that does not hold the full joint constraint, because the requester did not check every constraint, or because there were few constraints to check.

2.6 Token. The unit of text the model generates. The model produces one token at a time.

2.7 Conditioning. The process by which the model selects the next token. The model selects the highest-probability next token given the tokens already present. The tokens already present include the training data patterns, the supplied context, the supplied request, and the model's own earlier output in the same response.

2.8 Weighting term. One of the sources that shapes the next-token selection. This paper names four terms: the training data, the supplied context, the supplied request, and the model’s own earlier output. The output is a function of all four terms together.

2.9 Constraint cardinality. The number of constraints that must hold at the same time for a request. A request with few constraints has low cardinality. A request with many interacting constraints has high cardinality. Cardinality is not difficulty. A hard task with one constraint has low cardinality. An easy-looking task with many interacting constraints has high cardinality.

2.10 Transformation. The operation of taking a supplied input that already satisfies a joint constraint and producing an output that preserves that satisfaction while changing the form. In a transformation the joint constraint is already solved in the input. The model does not hold the joint constraint. The model operates on its frozen form.

3. Why the Full Goal Is Never an Input

This section states the first reason True Alignment is not reachable. The reason is that the requester’s full joint constraint is never a complete input to the process.

3.1 The output is a function of the four weighting terms in 2.8. The model selects each token from these terms. The model cannot condition on a constraint that is not present in these terms. A constraint that is not an input has no path to shape the output.

3.2 The requester’s full joint constraint is large and mostly unstated. A large interacting constraint set exists in the requester’s own model of the task. Most of it is never written into the request. No request of practical length transfers a large joint constraint completely. The requester writes a partial statement of the goal and holds the rest without stating it.

3.3 The training term does not contain the requester’s specific goal. The training data contains general patterns from many past tasks. It does not contain the requester’s particular joint constraint, because that joint constraint is specific to the requester’s particular work and was not part of the training data.

3.4 The context term contains only what the requester supplied. If the requester supplies material that embodies the joint constraint, the context term carries it. If the requester does not, the context term does not carry it. The context term is under the requester’s control and is bounded by what the requester chose to provide.

3.5 The consequence of 3.1 through 3.4 is that the full joint constraint is never fully present in any weighting term. The training term does not have it. The request term has only a partial statement of it. The context term has only what was supplied. The model’s own output term, discussed in Section 4, does not add it. There is no term that holds the complete goal.

3.6 Therefore the output cannot be a function of the full joint constraint, because the full joint constraint is not an input. The output can only be a function of the partial goal that

was supplied and the general patterns in the training data. True Alignment requires the output to hold the full joint constraint. The output cannot hold what it never received.

3.7 This limit is not a quality limit. A better model selects better tokens from the same four terms. It does not change which terms are present. A better model still cannot condition on a constraint that was never an input. The limit is structural. It is set by what the process takes in, not by how well the process runs.

4. All Tokens Are Equal: The Erosion of the Supplied Goal

Section 3 showed that the full goal is never a complete input. This section shows that the process also erodes the partial goal that was supplied. The two effects combine. The goal starts incomplete and then shrinks further as generation proceeds.

4.1 The model's own output re-enters as context. Each token the model generates is added to the tokens the next token conditions on. The model conditions on its own earlier output with the same weighting as any other context. The model does not treat the original request as a privileged term that overrides its own output.

4.2 This is the meaning of the phrase "all tokens are equal." A token from the original request and a token the model generated a moment ago are weighted by the same mechanism. Neither is marked as more authoritative. The next token is selected from the combined set without regard to which tokens came from the requester and which came from the model.

4.3 The request term is fixed in size. The requester writes the request once. It does not grow during generation. The model's own output term grows with every token. As generation proceeds, the model's own output becomes a larger share of the total context, and the fixed request becomes a smaller share.

4.4 The consequence of 4.3 is that the request's share of the weighting falls as generation continues. Early in a response the request is a large share of the context. Later in a response the model's own output is the larger share, and the request is a smaller share. The output is conditioned more on the model's own earlier tokens than on the original request.

4.5 A long generation does not move toward the request. It moves toward the direction its own early output set, because that early output is now the bulk of the context that later tokens condition on. Each generated token that drifts from the goal makes the next token condition on the drift. The drift compounds, because it is self-reinforcing through the context.

4.6 A special case of 4.5 is the restatement of intent. The model may generate an early paraphrase of the request. That paraphrase is now context. Later tokens condition on the paraphrase rather than on the original request, because the paraphrase is the nearest statement of the goal in the recent context. If the paraphrase dropped or changed a constraint, the rest of the output is faithful to the changed paraphrase, not to the original

request. There is no step that re-checks the output against the original request, because the original request is present only as one declining term.

4.7 Therefore the supplied partial goal is not held constant during generation. It is eroded by the model's own output, which grows and displaces it. True Alignment would require the full goal to be held throughout. The process cannot even hold the partial goal throughout, because the process overwrites it with its own output as it runs.

5. The Two Regimes: Low Cardinality and High Cardinality

Sections 3 and 4 gave the reasons True Alignment is not reachable. This section shows that no request size escapes those reasons. A low-cardinality request and a high-cardinality request both fail to reach True Alignment, for opposite reasons. There is no cardinality at which True Alignment is reached.

5.1 The low-cardinality regime. A low-cardinality request carries few constraints. The set of outputs that satisfy it is large. Many different outputs are all acceptable. The requester accepts one of them and has casual alignment.

5.2 A low-cardinality request does not define True Alignment. True Alignment requires a full joint constraint to hold. A low-cardinality request does not carry a full joint constraint. There is little to hold and little to violate. The output satisfies the request because the request asked for little, not because the output held a complete goal. The requester's acceptance is casual alignment, and it is not evidence of True Alignment, because there was no full joint constraint present to be aligned to.

5.3 A low-cardinality request usually produces an overfulfilled output. Because few constraints bind, the model fills the unspecified space with plausible content drawn from the training term. The output is often larger and more elaborate than the request. This overfulfillment is accepted when it lands inside the large acceptable set. It is casual alignment delivered by filling empty space, not True Alignment.

5.4 The high-cardinality regime. A high-cardinality request carries many interacting constraints. The set of outputs that satisfy it is small. Most outputs fail at least one constraint. The requester needs the output to hold the full joint constraint.

5.5 A high-cardinality request cannot be held jointly by the process. Section 3 showed the full joint constraint is never a complete input. Section 4 showed the supplied part is eroded by the model's own output. The more constraints the request carries, the more the model's own output must carry to hold them, and the more the growing output displaces the original request. High cardinality makes the erosion in Section 4 worse, because there is more to hold and the same mechanism drops it.

5.6 Therefore a high-cardinality request must be reduced to low cardinality, or it becomes a losing position. If the request is cut into small parts, each part is a low-cardinality request that the process can handle, and the requester holds the joint constraint across the parts. If the request is not cut down, the process is asked to hold a joint constraint it cannot

hold, and the output drifts, drops constraints, and compounds the drift. Pushing a high-cardinality request without cutting it down is a position that loses by structure, for three combined reasons stated next.

5.7 The three combined reasons. First, the full goal was never an input (Section 3), so the process starts without the complete constraint. Second, the model’s own output overwrites the supplied partial goal as generation proceeds (Section 4), so even the partial constraint is not held to the end. Third, all tokens are weighted equally (4.2), so more generated tokens mean more of the model’s own intent and less of the requester’s, which makes higher cardinality progressively harder rather than easier. These three reasons act together. They make high cardinality unreachable from above, while low cardinality never carried enough specification to define alignment from below.

5.8 The result is a two-sided closure. True Alignment is not reachable at low cardinality, because a low-cardinality request does not carry a full joint constraint to align to. True Alignment is not reachable at high cardinality, because the process cannot hold a full joint constraint against the erosion. There is no cardinality between them at which the process both receives a full joint constraint and holds it. True Alignment is unreachable across the whole range.

6. Building on What the Architecture Delivers

Sections 3 through 5 showed that True Alignment is not reachable. This section states what is reachable and how to use it. The claim is not that the architecture is useless. The claim is that the correct goal is transformation, not True Alignment, and that transformation is genuinely useful.

6.1 Transformation is reachable. In a transformation the joint constraint is already solved and frozen in the supplied input. The requester provides material that already satisfies the constraints. The model changes the form of that material while preserving the satisfaction. The model does not hold the joint constraint. It operates on the frozen form. The joint constraint lives in the supplied input, which is the context term, and the context term is one the process does condition on.

6.2 Transformation succeeds because it moves the constraint-holding outside the process. The requester, or the requester’s earlier work, holds the joint constraint. The supplied input carries it. The process is asked only to transform a thing that already satisfies the constraint. This is a low-cardinality operation over a high-cardinality artifact. The artifact embodies many constraints, but the process does not have to hold them, because they are already solved in the input.

6.3 Casual alignment is reachable by two paths. The first path is transformation, described in 6.1 and 6.2. The requester supplies material that holds the joint constraint, and the process transforms it. The second path is an accepted overfulfillment of a low-cardinality request, described in 5.3. The requester asks for little, the process fills the space, and the requester accepts the result. Both paths deliver casual alignment. Neither

delivers True Alignment. In the first path the constraint was held outside the process. In the second path there was little constraint to hold.

6.4 The working method. The method follows directly from the claim. It has four parts.

First, match the request to what the architecture delivers. Ask for transformation of supplied material, or ask a low-cardinality request, rather than asking the process to hold a large joint constraint.

Second, keep requests at low cardinality. If a request carries many interacting constraints, cut it into small parts, each of which is a low-cardinality request. Hold the joint constraint across the parts as the requester, not inside the process.

Third, supply the constraint as material to transform, not as a request to satisfy. When the constraint set is large, provide it as a working input that already satisfies the constraint, so the context term carries it and the process transforms it.

Fourth, treat rising cardinality as a signal to decompose, not a signal to push. When a request grows in interacting constraints, the correct response is to reduce it, because pushing it is the losing position in 5.6.

6.5 The method converts an unreachable goal into a reachable one. It does not attempt True Alignment. It builds on transformation and low-cardinality requests, which the architecture delivers. The requester gets a result that satisfies the requester, through a correct understanding of how the process works. This is the useful outcome the architecture supports, and it is reached by not asking the architecture for the outcome it cannot support.

7. Background: The Observations That Led to the Claim

The claim in this paper was reached from practitioner observation. The observations are given here as background. They are not the central argument. The central argument is the structural account in Sections 3 through 5. The observations are the data that made the structural account visible.

7.1 The training data has limited value for specific work. A practitioner working on an encapsulated project, in the practitioner's own code, with few libraries, found that the training term contributes little. The reason has two parts. First, the training data answers a different question than the specific request. A highly-rated public answer earned its rating against a general shared question. The practitioner's request is about a specific codebase that is not in the training data. The rating measures fit to the general question, and that fit is error relative to the specific request. Second, an encapsulated codebase defines its constraints internally. The correct output is determined by the practitioner's own earlier decisions, which are not in the training data. The training term therefore points at a target the request did not set.

7.2 The training data is thin at the expert level. The practitioner observed that public text is dense at the beginner level and thin at the expert level. Beginner questions are shared by many people, are asked and answered publicly, and appear many times in the

training data. Expert-level process knowledge is often protected as a commercial product, is often held as unwritten working knowledge, has a small audience, and branches into many specific cases with few instances each. The result is that the training term has little correct expert-level content to draw from. When an expert-level request is made, the highest-probability output is drawn from the dense beginner-level content, and it is presented at full confidence. The absence of the expert answer in the training data does not produce a signal. It produces a beginner-level output in the shape of an expert one.

7.3 The same behavior has different severity in different regimes. The practitioner compared two projects. One is a game engine with a very large joint constraint, including the placement of threads on specific processor cores. The other is a local markdown viewer server with a small constraint set. In the markdown viewer the process added a feature the practitioner did not request. The practitioner accepted it, because the viewer is low-cardinality and the addition landed inside the large acceptable set. The practitioner stated that the same unrequested addition in the game engine session would be grounds to stop the session at once, because the engine is high-cardinality, the addition enters an interacting joint constraint, and the addition then becomes context that the rest of the session conditions on and compounds. The behavior is identical. The severity is set by the cardinality of the context the behavior enters. This observation supports the two-regime account in Section 5.

7.4 How the observations led to the claim. The observations in 7.1 through 7.3 share a cause. The training term does not carry the requester’s specific goal, whether because the goal is a different target (7.1), because the goal is absent from public text (7.2), or because the goal is a large interacting constraint the process cannot hold (7.3). This shared cause is the structural account in Section 3: the full goal is never an input. The observations are the practitioner’s route to the claim. The claim itself rests on the structure, not on the observations, and the observations are provided here as the background that made the structure visible.

8. Scope Conditions

The boundaries of the claim are stated here.

8.1 The claim concerns the current architecture. The architecture is sequential token generation that conditions on training, context, request, and its own prior output, without a step that holds and jointly checks a full constraint set. The claim follows from this architecture. The paper does not claim the limit is permanent across all possible future architectures.

8.2 The claim concerns True Alignment as defined in 2.3. It states that the output holding the requester’s full joint constraint is not reachable. It does not state that the output is useless. It states that a different and weaker condition, casual alignment, is reachable and useful.

8.3 The claim does not assign fault. It does not state that the model is defective. It does not state that any developer made an error. It states how the architecture works and what

follows from that. The correct response to the claim is to use the architecture for what it delivers, which is transformation and low-cardinality requests.

8.4 The claim depends on the definition of alignment in 2.3, which is the strong definition. A reader who uses the weak definition, casual alignment, would say that accepted output is aligned. The paper does not contest that the weak condition is reachable. It states that the strong condition is not, and that treating the weak condition as if it were the strong one is the source of the mistaken expectation that the process holds a goal it does not hold.

9. Summary

True Alignment, defined as the output holding the requester’s full joint constraint, is not reachable with the current architecture. The reason is structural and has two parts. First, the full joint constraint is never a complete input to the process, because the request states only part of it and the training data does not contain the requester’s specific goal. Second, the process erodes even the partial goal that was supplied, because the model’s own output re-enters as context with equal weighting and displaces the original request as generation proceeds. These two parts close both regimes. A low-cardinality request never carries a full joint constraint to align to. A high-cardinality request cannot be held against the erosion and must be cut to low cardinality or it becomes a losing position. There is no cardinality at which the full goal is both received and held.

What is reachable is casual alignment, where the requester accepts the output. It is reached by two paths: transformation of supplied material that already holds the joint constraint, and accepted overfulfillment of a low-cardinality request. Both are useful. Neither is True Alignment. The correct use of the architecture is therefore transformation, not the pursuit of a goal the process cannot hold. The requester who matches the request to what the architecture delivers, keeps requests at low cardinality, supplies large constraints as material to transform, and decomposes rising cardinality rather than pushing it, receives a result that satisfies the requester through a correct understanding of the process. True Alignment is not the goal this architecture can meet. Transformation is, and it is enough.

Appendices

Appendix A. The two definitions of alignment stated side by side, with the two-side property and a table of which weighting terms carry the goal in each case.

Appendix B. The two-regime closure of Section 5 as a single table: low cardinality underspecifies, high cardinality overwhelms, and the cause of failure in each.

Appendix C. The working method of Section 6.4 as a standalone checklist.

Appendix D. The background observations of Section 7 as data points, each mapped to the structural reason in Section 3 it supports.

Appendices — HOWL-LLM-11-2026

These appendices carry material that supports the body without repeating it. Each table brings in a point that was reasoned through but not stated in full in Sections 1 through 9, and connects it to the structural claim.

Appendix A — The Two Definitions Side by Side

The body defines True Alignment and casual alignment and states that they are measured on different sides (2.5). This appendix carries the material the body did not: which weighting term holds the goal in each reachable case, and why the requester cannot observe the difference from the accept decision alone.

Table A.1 — Where the goal lives in each case

Property	True Alignment	Casual alignment via transformation	Casual alignment via low-cardinality overfulfillment
What holds the joint constraint	The process, inside the decode	The supplied input, outside the process	Nothing; there is little constraint to hold
Which weighting term carries it	None; the full constraint is never an input	The context term (the supplied working material)	Neither; the request term is thin and the rest is training-term fill
Measured on which side	The output against the full goal	The output against the supplied input	The requester's acceptance
Reachable	No	Yes	Yes
Failure signature when it goes wrong	n/a; not reachable	Bounded transform error, readable against the input	Overfulfillment lands outside the acceptable set; visible on inspection
What the requester checks at accept time	Would have to check the full joint constraint	Checks the transform against the input	Checks that the result looks acceptable

The connecting point: the requester's accept decision observes casual alignment in both reachable columns and never observes True Alignment in any column. The accept decision is a check against what the requester can see, which is the output and, in the transform case, the input. It is not a check against the full joint constraint, because most of that constraint is unstated and lives only in the requester's own model. This is why acceptance is not evidence of True Alignment: acceptance is measured on a side that does not contain

the full goal.

Table A.2 — The two-side property made concrete

This table brings in the “make Doom, but better” case, which the body referenced only in passing, and pairs it with the transform case to show the two sides diverging.

Case	Requester’s stated request	Requester’s full joint constraint	Output	Casual alignment	True Alignment
“Make Doom, but better”	Very thin; a few words	Large and entirely unstated; the requester has a rich but unexpressed sense of “better”	A playable game the requester enjoys	Present; the requester is happy	Absent; the output holds none of the unstated joint constraint, because none of it was an input
Transform a working source	Convert this working module to the new form	Already solved and frozen in the supplied source	The module in the new form, preserving behavior	Present; the requester accepts the transform	Absent; the process held nothing, the input held the constraint

The connecting point: in the first row the requester is happy and the full goal was never held, which is the clearest demonstration that casual alignment and True Alignment are independent. A happy requester on a thin request is the case where the two definitions are furthest apart, because the thinner the request, the larger the unstated remainder, and the larger the gap between “accepted” and “held the full goal.”

Appendix B — The Two-Regime Closure

The body states that neither low nor high cardinality reaches True Alignment (Section 5). This appendix states the closure as one table and brings in the mechanism that connects the two regimes, which the body stated in parts across 5.5 through 5.7.

Table B.1 — Why each regime fails, from opposite directions

	Low-cardinality regime	High-cardinality regime
Constraint supplied	Too little to define the full goal	More than the process can hold
Acceptable output set	Large	Small
What the process does	Fills the unspecified space from the training term	Drops constraints as its own output displaces the request
Why True Alignment fails	There was no full joint constraint present to align to	The full joint constraint was present in part but could not be held
Direction of failure	From below: never enough specification	From above: too much for the process to carry
Typical delivered result	Accepted overfulfillment	Drift, dropped constraints, compounding
Correct response	Accept it as casual alignment if it lands in the set	Cut to low cardinality, or supply as material to transform

Table B.2 — The three combined reasons that make high cardinality a losing position

The body listed these three reasons in 5.7 but did not show how each one worsens as cardinality rises. This table brings in that scaling relationship.

Reason	Stated in body as	How it scales with rising cardinality
The full goal was never an input	Section 3	More constraints means a larger unstated remainder, so the share of the goal that reaches the process falls as cardinality rises
The model's own output overwrites the supplied goal	Section 4	More constraints require a longer output to address, and a longer output means more self-tokens displacing the request, so erosion grows with cardinality
All tokens are weighted equally	4.2	More self-tokens carry more of the model's own intent at equal weight, so the request's share falls faster the more the process generates

The connecting point: the three reasons are not independent additive costs. They compound. A higher-cardinality request produces a longer output, a longer output produces more self-tokens, more self-tokens at equal weight displace more of the request, and the larger unstated remainder means there was less of the goal present to begin with. This is why cardinality does not merely make the task harder in proportion; it makes True Alignment recede faster than the request grows.

Appendix C — The Working Method as a Checklist

The body states the method in prose (6.4). This appendix states it as an ordered procedure and brings in the point-of-application for each item, which the body did not give. The order is the order of a task’s lifecycle.

Table C.1 — Before the request: sizing and routing

Action	What it prevents	When to apply
Estimate the cardinality before writing the request	Asking the process to hold a joint constraint it cannot hold	Any request with more than a small number of interacting constraints
Route large-constraint work to transformation	Requiring the decode to hold the goal instead of the input	Whenever a working source exists that already satisfies the constraint
Cut a high-cardinality request into low-cardinality parts	Drift and dropped constraints from over-large requests	When the cardinality estimate is above the level a single request can carry
Hold the joint constraint as the requester, across the parts	Loss of the goal in the gaps between parts	Whenever a request was cut into parts
Supply the constraint as material, not as a request to satisfy	The training term substituting a general target for the specific goal	When the goal is specific to the requester’s own work and absent from public patterns

Table C.2 — During generation: watching the erosion

Action	What it detects	Observable that triggers it
Read the output against the original request, not against the output’s own restatement	The request being displaced by an early paraphrase	Any restatement of the goal appearing in the output
Compare delivered scope to requested scope	Unrequested additions entering the output	Output broader or longer than the request

Action	What it detects	Observable that triggers it
Stop the session on an unrequested addition in a high-cardinality context	The addition becoming context that the rest of the session compounds	An addition the request did not call for, in work with a large joint constraint
Accept an unrequested addition in a low-cardinality context	Wasted effort correcting a harmless addition	An addition that lands inside a large acceptable set

The connecting point brought in here: the same observable, an unrequested addition, triggers opposite actions in the two regimes. In high cardinality it triggers stopping the session, because the addition will compound. In low cardinality it triggers acceptance, because the addition is absorbed. The trigger is not the addition. The trigger is the addition together with the cardinality of the context it entered. This is the severity-is-context-dependent result from the background, applied as an operating rule.

Table C.3 — On drift: the decision that dominates the outcome

Action	Against	Test to run
Test whether re-supplying the goal realigns the output	Continuing to push a session that has already drifted past recovery	Re-supply a constraint known to be correct; does the output realign?
If it realigns, continue	Abandoning a recoverable session	The output returns to the goal after one re-supply
If it does not realign, stop and restart	Adding more context to a session already dominated by drift	The output does not return to the goal after re-supply
Restart with the goal supplied from the first token	Repeating the drift that the accumulated context already produced	Was the drift present from early in the session, or introduced by accumulation?

The connecting point: the decision on drift is not whether the current output is correct. It is whether the session can still be corrected. A session that realigns after one re-supply is in a different state than one that does not, and the two states require opposite actions. Continuing a session that cannot be corrected adds more self-tokens to a context already dominated by the model's own drift, which is the losing position from 5.7 applied within a single session rather than across a request.

Appendix D — Background Observations as Data Points

The body gives the background observations in prose (Section 7) and states that they share one cause. This appendix states each as a data point and connects it to the structural reason it supports. It brings in the two distinct ways the training term fails, which the body stated but did not separate into a table, and adds the point about the interface cost that did not reach the body at all.

Table D.1 — Each observation mapped to the structural reason

Observation	What was seen	Structural reason it supports	Why the reason predicts it
Training term is a different target	For encapsulated own-code work, public answers point at a general question the request did not ask	The full goal is never an input (Section 3)	The training term holds general targets, not the requester’s specific joint constraint, so it substitutes the general for the specific
Training term is empty at the expert level	Public text is dense at the beginner level and thin at the expert level	The full goal is never an input (Section 3)	Expert knowledge is protected, unwritten, low-audience, and branched, so it is absent from the training term, which then draws the nearest beginner-level neighbor
Same behavior, different severity	An unrequested addition was harmless in a low-cardinality viewer and would stop a high-cardinality engine session	The two-regime closure (Section 5)	Severity is set by the cardinality of the context the behavior enters, because in high cardinality the addition compounds through the interacting joint

Table D.2 — The two distinct failures of the training term

The body treated the training term as weak for specific work but did not separate the two ways it is weak. They have different signatures and are brought in here.

	Different-target failure	Absent-target failure
What is wrong	The training term has content, aimed at the wrong target	The training term has little or no correct content for the request
Where it occurs	Encapsulated own-code work, where the goal is specific and internal	Expert-level work, where the answer is protected or unwritten
What the process does	Draws a confident output aimed at the general target	Draws a confident output from the nearest beginner-level content
Signature	Output looks plausible but solves a slightly different problem	Output looks authoritative but is beginner-level under an expert-shaped surface
Why it is silent	The wrong target is close enough to look right	The absence of the answer produces a neighbor, not a signal

The connecting point: both failures come from the same structural fact, that the full goal is never an input, but they present differently and are detected differently. The different-target failure is caught by reading the output against the specific request. The absent-target failure is caught only by an expert who knows the real answer, because the requester who needed the answer cannot judge it. This is why the two are separated: the detection method differs even though the cause is shared.

Table D.3 — The interface cost, brought in from the reasoning

This point was reasoned through and did not reach the body. It connects the impossibility claim to a cost the requester pays that is produced by the presentation, not by the process.

Element	Statement
What it is	The process presents its output in a conversational form that resembles a collaborator
Why it matters here	A failure to reach True Alignment is presented as a collaborator's mistake rather than as a plain limit of a transform tool

Element	Statement
The cost	The requester spends effort responding to the failure as if it were a person's error, which is effort a non-conversational tool would not demand
Connection to the claim	The impossibility of True Alignment is structural and impersonal, but the conversational surface presents each instance of it as a personal lapse, which adds a cost that belongs to the presentation and not to the structure
Correct response	Read the failure as a property of the architecture, not as a lapse to be discussed, and route the work to transformation rather than to further conversation

The connecting point: the interface cost is the one cost in this paper that is not caused by the token mechanism. It is caused by the framing of the output as conversation. It is included because it changes how a requester responds to the impossibility. A requester who understands the impossibility as structural stops trying to correct the process into True Alignment through conversation, which is effort spent against a limit that conversation cannot move.

[[HOWL-LLM-11-2026](#)] True Alignment is Impossible. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM-11-2026>

[[HOWL-LLM-10-2026](#)] The Cardinality Ceiling. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HO-LLM-10-2026>